

Formal Analysis and Supply Chain Security for Agentic AI Skills

- 现有验证思路都是启发式的
- 形式化验证了Skills的安全性
- SkillFortify除了检验当前Skill的安全性，还会检查其依赖Skill的安全性（一条链上所有都安全才是安全的）
- 文中的几个判断方法

- Dolev-Yao模型

- 假设有一个能力最强的敌手，如果能防御住他的攻击，那么我就可以预防所有的攻击

Theorem 3.6

- Capability Lattice能力晶格

- 将权限分为四级 ($NONE < READ < WRITE < ADMIN$)
 - 针对文件系统、网络、环境变量、Shell、技能调用等 8 个核心维度
 - 系统扫描代码，推导出需要的权限，eg. 识别 HTTP 写入方法推导出网络写入权限

引入zkVM，摄入闭源的Agent技能代码，输出 `Required_Capabilities`，并断言 `assert(Required_Capabilities ≤ Declared_Capabilities)`；输出ZK Receipt，用户可以看到“该技能哈希值为 X ，声明权限为 Y ，且已通过扫描”

- 如果一个技能声称的权限低于分析出的所需权限，那么判定为违规
- 依赖链条的防御
 - 代理依赖图ADG，在依赖图里加入安全能力约束，子技能的权限不能超过父技能
 - SAT求解：将依赖解析问题转化为布尔可满足问题SAT并求解

电路外求解，电路上验证：链下跑普通的SAT，得到Lockfile；ZK电路遍历ADG依赖树，验证是否满足 `Child_Capability ≤ Parent_Capability`，并验证所有依赖库的版本哈希

- 确定性锁文件Lockfile（包含了依赖关系和每个节点的权限声明）确保所有环境下下载的内容是一致的
- 信任量化成得分
 - 基于是否有签名和验证、静态分析是否通过、社区活跃度、是否有漏洞的历史记录来多维度计算得分
 - 木桶效应：最终得分依赖于依赖链中最弱的一环，通过乘法传播
 - 时间衰减：如果Skill无人维护更新，会随着时间衰减；但是更新以后也需要重新验证

就是找最小值和乘法运算等，电路输出值，例如 `Proof(Trust_Score ≥ 0.75)`，即证明该技能达到了“L3 级别”，但不向外界透露它是靠出处签名拿的高分，还是靠社区活跃度拿的高分。

- 补充内容一定会提升得分，不修改一定会降低得分

- **开发者 (Prover)** : 拥有技能的明文代码, 负责在本地运行 zkVM 和 SAT 求解器, 生成 ZK Proof 和明文的 Lockfile。
- **去中心化注册表 / 智能合约 (Verifier)** : 维护一个映射表 `Skill_Hash -> (Declared_Capabilities, ZK_Proof)`。它只运行极低成本的 `verify(proof, public_inputs)` 函数。
- **Agent 调用方 (User)** : 在拉取技能时, 查验注册表上的 ZK 证明。只要验证通过, 就可以放心地将自己的私钥或 API Key 交给这个闭源技能。